

Pretentious Self-Parsing and Self-Evaluation of λ -Calculus

Heewon Lee

School of Computing, KAIST

October 15, 2026

Motivation

- Reading strings of λ -expressions and evaluating them is itself a computational task.

Motivation

- Reading strings of λ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computatable task can be performed by a λ -expression.

Motivation

- Reading strings of λ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computable task can be performed by a λ -expression.
- $\Rightarrow$ Lambda evaluation should also be possible with a λ -expression!

Motivation

- Reading strings of λ -expressions and evaluating them is itself a computational task.
- Church-Turing thesis: Any computable task can be performed by a λ -expression.
- $\Rightarrow$ Lambda evaluation should also be possible with a λ -expression!

$\therefore$ Goal: Parse and evaluate λ -expressions using λ -calculus!

Concrete and Abstract Syntax

Concrete Syntax

Abstract Syntax

Concrete and Abstract Syntax

Concrete Syntax

$\langle \text{term} \rangle ::= \langle \text{nat} \rangle$ (Variable)
| $(\lambda. \langle \text{term} \rangle)$ (Function)
| $((\langle \text{term} \rangle) (\langle \text{term} \rangle))$ (Application)

Abstract Syntax

Concrete and Abstract Syntax

Concrete Syntax

$\langle \text{term} \rangle ::= \langle \text{nat} \rangle$	(Variable)
$\quad (\lambda. \langle \text{term} \rangle)$	(Function)
$\quad ((\langle \text{term} \rangle) (\langle \text{term} \rangle))$	(Application)

Abstract Syntax

$t ::= \text{Var}(n)$	(Variable)
$\quad \text{Lam}(t)$	(Function)
$\quad \text{App}(t, t)$	(Application)

Concrete and Abstract Syntax

Concrete Syntax

$\langle \text{term} \rangle ::= \langle \text{nat} \rangle$	(Variable)
$\quad (\lambda. \langle \text{term} \rangle)$	(Function)
$\quad ((\langle \text{term} \rangle) (\langle \text{term} \rangle))$	(Application)

Abstract Syntax

$t ::= \text{Var}(n)$	(Variable)
$\quad \text{Lam}(t)$	(Function)
$\quad \text{App}(t, t)$	(Application)

e.g.) “ $(\lambda. 0)$ ” $\rightarrow$ [LPAR; LAM; NAT(0); LPAR] $\rightarrow$ λ -repr. of tokens $\rightarrow$ λ -repr. of AST $\rightarrow$ λ -expr.

Concrete and Abstract Syntax

Concrete Syntax

$\langle \text{term} \rangle ::= \langle \text{nat} \rangle$	(Variable)
$\quad (\lambda. \langle \text{term} \rangle)$	(Function)
$\quad ((\langle \text{term} \rangle) (\langle \text{term} \rangle))$	(Application)

Abstract Syntax

$t ::= \text{Var}(n)$	(Variable)
$\quad \text{Lam}(t)$	(Function)
$\quad \text{App}(t, t)$	(Application)

e.g.) “ $(\lambda. 0)$ ” $\rightarrow$ [LPAR; LAM; NAT(0); LPAR] $\rightarrow$ λ -repr. of tokens $\rightarrow$ λ -repr. of AST $\rightarrow$ λ -expr.

Concrete and Abstract Syntax

Concrete Syntax

$\langle \text{term} \rangle ::= \langle \text{nat} \rangle$	(Variable)
$\quad (\lambda. \langle \text{term} \rangle)$	(Function)
$\quad ((\langle \text{term} \rangle) (\langle \text{term} \rangle))$	(Application)

Abstract Syntax

$t ::= \text{Var}(n)$	(Variable)
$\quad \text{Lam}(t)$	(Function)
$\quad \text{App}(t, t)$	(Application)

e.g.) “ $(\lambda. 0)$ ” $\rightarrow$ [LPAR; LAM; NAT(0); LPAR] $\rightarrow$ λ -repr. of tokens $\rightarrow$ λ -repr. of AST $\rightarrow$ λ -expr.

- 1 Represent each token
- 2 Represent each AST constructor
- 3 Parser `e_parse` from token list to AST
- 4 Evaluator `e_eval` from AST to λ -expression

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data types (ADTs): Sum, product, and recursive types

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data types (ADTs): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data types (ADTs): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
$\text{Var}(n)$	$\lambda v / a. v \ n$
$\text{Lam}(t)$	$\lambda v / a. \lambda t$
$\text{App}(t_1, t_2)$	$\lambda v / a. a \ t_1 \ t_2$
LPAR	$\lambda / r \ a \ v. l$
RPAR	$\lambda / r \ a \ v. r$
LAM	$\lambda / r \ a \ v. a$
$\text{NAT}(n)$	$\lambda / r \ a \ v. v \ n$

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data types (ADTs): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
-------------	----------------

$\text{Var}(n)$	$\lambda v / a. v \ n$
-----------------	------------------------

$\text{Lam}(t)$	$\lambda v / a. / \ t$
-----------------	------------------------

$\text{App}(t_1, t_2)$	$\lambda v / a. a \ \mathbf{t_1} \ \mathbf{t_2}$
------------------------	--

LPAR	$\lambda / r \ a \ v. /$
------	--------------------------

RPAR	$\lambda / r \ a \ v. r$
------	--------------------------

LAM	$\lambda / r \ a \ v. a$
-----	--------------------------

NAT(n)	$\lambda / r \ a \ v. v \ n$
-----------------	------------------------------

Case statement

$$t \ fv \ fl \ fa = \begin{cases} fv \ n & (t = \text{Var}(n)) \\ fl \ t' & (t = \text{Lam}(t')) \\ fa \ t_1 \ t_2 & (t = \text{App}(t_1, t_2)) \end{cases}$$

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data types (ADTs): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
-------------	----------------

$\text{Var}(n)$	$\lambda v / a. v \ n$
-----------------	------------------------

$\text{Lam}(t)$	$\lambda v / a. \lambda t$
-----------------	----------------------------

$\text{App}(t_1, t_2)$	$\lambda v / a. a \ \mathbf{t_1} \ \mathbf{t_2}$
------------------------	--

LPAR	$\lambda / r \ a \ v. \lambda$
------	--------------------------------

RPAR	$\lambda / r \ a \ v. r$
------	--------------------------

LAM	$\lambda / r \ a \ v. a$
-----	--------------------------

$\text{NAT}(n)$	$\lambda / r \ a \ v. v \ n$
-----------------	------------------------------

Case statement

$$t \ fv \ fl \ fa = \begin{cases} fv \ n & (t = \text{Var}(n)) \\ fl \ t' & (t = \text{Lam}(t')) \\ fa \ t_1 \ t_2 & (t = \text{App}(t_1, t_2)) \end{cases}$$

Example: $\lambda x. \lambda y. x$

$\lambda v / a. \lambda$

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data types (ADTs): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
-------------	----------------

Var(n)	$\lambda v / a. v \ n$
------------	------------------------

Lam(t)	$\lambda v / a. / \ t$
------------	------------------------

App(t_1, t_2)	$\lambda v / a. a \ \mathbf{t_1} \ \mathbf{t_2}$
-------------------	--

LPAR	$\lambda / r \ a \ v. /$
------	--------------------------

RPAR	$\lambda / r \ a \ v. r$
------	--------------------------

LAM	$\lambda / r \ a \ v. a$
-----	--------------------------

NAT(n)	$\lambda / r \ a \ v. v \ n$
------------	------------------------------

Case statement

$$t \ fv \ fl \ fa = \begin{cases} fv \ n & (t = \text{Var}(n)) \\ fl \ t' & (t = \text{Lam}(t')) \\ fa \ t_1 \ t_2 & (t = \text{App}(t_1, t_2)) \end{cases}$$

Example: $\lambda x. \lambda y. x$

$\lambda v / a. / (\lambda v / a. /$)

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data types (ADTs): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
-------------	----------------

Var(n)	$\lambda v / a. v \ n$
------------	------------------------

Lam(t)	$\lambda v / a. \lambda t$
------------	----------------------------

App(t_1, t_2)	$\lambda v / a. a \ \mathbf{t_1} \ \mathbf{t_2}$
-------------------	--

LPAR	$\lambda / r \ a \ v. \lambda$
------	--------------------------------

RPAR	$\lambda / r \ a \ v. r$
------	--------------------------

LAM	$\lambda / r \ a \ v. a$
-----	--------------------------

NAT(n)	$\lambda / r \ a \ v. v \ n$
------------	------------------------------

Case statement

$$t \ fv \ fl \ fa = \begin{cases} fv \ n & (t = \text{Var}(n)) \\ fl \ t' & (t = \text{Lam}(t')) \\ fa \ t_1 \ t_2 & (t = \text{App}(t_1, t_2)) \end{cases}$$

Example: $\lambda x. \lambda y. x$

$$\lambda v / a. \lambda (\lambda v / a. \lambda (\lambda v / a. a \quad))$$

Theory: Algebraic Data Types and Scott Encoding

- Algebraic data types (ADTs): Sum, product, and recursive types
- Scott encoding: General encoding of ADTs

Constructor	Scott Encoding
-------------	----------------

$\text{Var}(n)$	$\lambda v / a. v \ n$
-----------------	------------------------

$\text{Lam}(t)$	$\lambda v / a. \lambda t$
-----------------	----------------------------

$\text{App}(t_1, t_2)$	$\lambda v / a. a \ \mathbf{t_1} \ \mathbf{t_2}$
------------------------	--

LPAR	$\lambda / r \ a \ v. \lambda$
------	--------------------------------

RPAR	$\lambda / r \ a \ v. r$
------	--------------------------

LAM	$\lambda / r \ a \ v. a$
-----	--------------------------

NAT(n)	$\lambda / r \ a \ v. v \ n$
-----------------	------------------------------

Case statement

$$t \ fv \ fl \ fa = \begin{cases} fv \ n & (t = \text{Var}(n)) \\ fl \ t' & (t = \text{Lam}(t')) \\ fa \ t_1 \ t_2 & (t = \text{App}(t_1, t_2)) \end{cases}$$

Example: $\lambda x. \lambda y. x$

$$\lambda v / a. \lambda (\lambda v / a. \lambda (\lambda v / a. a (\lambda s z. s \ z)))$$

Theory: Church vs. Scott Encoding